



Applied Machine Learning for Industrial Data Analysis

ARJUN KRISHNA KRISHNAKUMAR

Student No: R00276690

arjunkrishna.krishnakumar@mymtu.ie

Module code: STAT9013

Table of content

Abstract.....	2
1. Introduction.....	3
2. Related Work.....	4
3. Methodology.....	5
3.1 Dataset.....	5
3.2 Preprocessing Pipeline.....	5
3.3 Dimensionality Reduction : PCA.....	6
3.4 Clustering.....	6
3.5 Classification.....	6
4. Experiments and Results.....	7
4.1 Data Understanding and Preprocessing.....	7
4.2 Exploratory Data Analysis (EDA).....	9
4.3 Outlier Analysis.....	11
4.4 Dimensionality Reduction : PCA.....	12
4.5 Clustering.....	16
4.6 Classification.....	18
4.7 Model Evaluation.....	20
5. Discussion.....	23
5.1 Critical Analysis of Results.....	23
5.2 SMOTE and Optimism Bias.....	23
5.3 Limitations.....	24
6. Conclusion.....	25
7. References.....	26
8. Appendix.....	27

Abstract

This report is about a comprehensive machine learning analysis applied to the SECOM Semiconductor Manufacturing Dataset, which serves as a benchmark in predictive maintenance and quality control in the modern manufacturing process. The dataset is high dimensional real-world industrial dataset comprises 1567 features with a binary quality outcome; PASS or FAIL. The study follows an end-to-end Machine learning pipeline which comprises data preprocessing with missing-value handling , standard scaler. Exploratory data analysis with visualisations, outlier detection via z-score and IQR methods, dimensionality reduction using Principle component Analysis, unsupervised grouping using K-Means and Hierarchical Clustering, and supervised classification using k-Nearest Neighbours (k-NN), Decision Tree, and Support Vector Machine (SVM).

To address the significant class imbalance where only 6.6% of units fail, SMOTE oversampling was used to help the models to better identify these rare cases. Among the techniques used, Support Vector machining (SVM) achieved the highest accuracy of 93% and had the best F1-macro score of 0.52. This report critically evaluates all modeling choices and hyperparameters, even suggesting alternatives in some instances, with an emphasis on the challenge of detecting rare failures.

1. Introduction

Semiconductor manufacturing is one of the most data-intensive sectors of modern engineering. Hundreds of sensors dynamically work together to monitor process parameters such as temperature, pressure, chemical concentrations and timing signals, throughout production, resulting in thousands of data collected for each wafer. Even slight deviations in these parameters can result in defects that are only detected at final quality inspection, often after significant value has been added. Early detection of these failures using sensor data, which is collected during manufacturing, represents a key opportunity in waste reduction, cost reduction and improving the yield. In this assignment a complete machine learning pipeline is used in SECOM Manufacturing Dataset, which is a publicly available high dimensional real industry dataset, which reflects the real challenges that pop up in real-world production. Extreme high-dimensionality (590 sensors), substantial missing data (up to 100% for some sensors), severe class imbalance (only 6.6% failures), and significant noise.

The goal of the study includes;

- To systematically preprocess and clean the raw sensor data.
- Explore data in depth to understand the dataset structure and class separation.
- To detect and analyse outliers.
- Reduce dimensionality by PCA with principled component selection.
- To apply an unsupervised clustering approach and interpret the results.
- Train and tune three classifiers namely k-NN, Decision Tree, SVM.
- Evaluate and compare all models critically on failure detection performance.

The entire analysis was done using Python libraries like scikit-learn, pandas, NumPy, matplotlib and seaborn, and also used in imbalanced-learn. The full implementation is included in the accompanying Jupyter Notebook.

2. Related Work

McCann and Johnston [1] framed the SECOM dataset as a high-dimensional fault detection problem where extreme class imbalance and a high feature-to-sample ratio are the two primary obstacles, establishing that both must be addressed before any classifier can perform reliably. Jolliffe [2] provided the theoretical basis for Principal Component Analysis, and retaining components at the 95% explained variance threshold has since become a standard practice in industrial applications, offering a balance between information preservation and computational efficiency while stabilising classifiers in high-dimensional spaces.

Chawla et al. introduced SMOTE to address class imbalance by generating synthetic minority samples through interpolation in feature space, improving minority-class recall compared to unbalanced training [3]. Douzas et al. [4] later showed that applying SMOTE before cross-validation splits causes data leakage, inflating validation recall estimates, and recommended applying it within each fold instead. This is directly relevant to the inflated cross-validation recall observed in this project. Cortes and Vapnik [5] introduced SVMs, and Hsu et al. [6] demonstrated that jointly tuning the regularisation parameter C and kernel coefficient γ via systematic search is essential for reliable RBF kernel performance. Jolliffe provided the theoretical basis for Principal Component Analysis, and retaining components at the 95% explained variance threshold has since become a standard practice in industrial applications, offering a balance between information preservation and computational efficiency while stabilising classifiers in high-dimensional spaces. Decision Trees offer interpretability through rule-based structure but are sensitive to noise, while k -NN degrades in high-dimensional spaces due to distance concentration, making PCA reduction a prerequisite for its effective use.

3. Methodology

3.1 Dataset

The SECOM Manufacturing Dataset is used for the study, which is downloaded from the UCI Machine Learning Repository (ID 179). The dataset consists of two files, namely `secom.data`, which contains 1,567 rows and 590 sensor columns separated by whitespace, and `secom_labels.data`, which contains the binary outcome column and corresponding timestamps. The original labels were encoded as 1 for Fail and -1 for Pass. For this study, the labels were converted into a binary format where 1 represents Fail and 0 represents Pass to simplify the classification process.

3.2 Preprocessing Pipeline

Preprocessing followed four sequential steps, each justified by characteristics of the data: • Features with only one unique value (including NaN) carry no information and were dropped first. Preprocessing was performed through four sequential steps based on characteristics of data. Features with unique values are removed in the first steps, since they carry no information. Followed by missing value removal, a threshold of 50% was initialised and features missing beyond this are removed. This threshold was chosen because imputing the majority of a column introduces more imputation artefact than signal. Of the 590 original features, a substantial subset exceeded this threshold. The remaining missing values were median imputation where it gets filled with the column median rather than the mean, because several sensor distributions showed heavy skew and extreme outliers that could potentially distort the results. After median imputation in the missing values, zero-variance columns were dropped because those features contained no discriminative power and the dataset had 446 features. Since PCA, k-NN and SVM are highly sensitive to differences in feature scale, the remaining features were standardised to zero mean and unit variance using `StandardScaler`.

3.3 Dimensionality Reduction : PCA

PCA was applied on the feature matrix, which was scaled before applying PCA. Initially PCA with all components was fitted against the explained variance spectrum. The component counts were determined from the cumulative explained variance curve to capture 90%, 95% and 99% of total variance. The PCA model retaining 95% of the total variance was used for all remaining steps, as discussed further in Section 4.4. A separate two-component PCA was also created for 2D visualisation purposes.

3.4 Clustering

PCA was used to reduce the features followed by K-Means clustering. The number of clusters k was determined using inertia (elbow method) and silhouette score from $k = 2$ to 10, The silhouette score was preferred as the primary criterion the elbow method can be visually ambiguous. A dendrogram produced by applying hierarchical clustering with Ward linkage to a 250-sample subsample of the data (using the first 15 PCs) visually corroborated both flat K-Means partitions.

3.5 Classification

Classification models such as k-Nearest Neighbours (k-NN), Decision Tree, and Support Vector Machine (SVM) are used for semiconductor fault detection. The models were fitted on the SMOTE-balanced training data and then evaluated on the original imbalanced test set to better simulate manufacturing conditions. As with the SMOTE technique there is a risk of data leakage, it was done after creating train and test sets.

All models are tuned using 5-fold cross-validation hyperparameter tuning. For k-NN, GridSearchCV was employed to try different values of k ranging from 3 to 29 for both uniform and distance-based weighting with optimization on Recall for the fail class. The Decision Tree model is tuned using different `max_tree_depth`, `min_samples_leaf`, and splitting criteria (gini or entropy), but fixed `class_weight='balanced'` and maximized macro F1-score. For the SVM model, a RandomizedSearchCV over 30 iterations using search space of different levels of c , kernel types (rbf and sigmoid) and other parameters were employed via a log-uniform distribution to optimise macro F1-score with balanced class weighting.

4. Experiments and Results

4.1 Data Understanding and Preprocessing

The raw dataset contained 1,567 samples and 590 sensor features. Inspection of missing data revealed that 469 features had at least one missing value, and a large proportion of those were missing in 30–80% of rows. Below table shows the overall summary of the dataset along with the missing values.

Property	Value
Total samples	1567
Total features	590
Pass samples	1,463 (93.4%)
Fail samples	104 (6.6%)
Sensors with missing values	538
Sensors with more than 50% missing	28
Total missing cells	41,951

Table 1. SECOM dataset summary

Figure 1 shows the top 40 sensors by missing percentage, indicating a clear group of sensors well above the 50% threshold shown as a dashed red line. After removing constant features, high missing features, and after imputation of zero-variance features, the dataset was reduced to 446 features having a 20% reduction. The class distribution was strongly imbalanced consisting of 1,463 pass samples (93.4%) versus 104 fail samples (6.6%). This imbalance is typical of semiconductor manufacturing where the majority of units pass. It means that a naive classifier that always predicts 'Pass' achieves 93.4% accuracy while

detecting zero failures making accuracy a misleading metric for this task. In this process, finding every actual failure is the main task.

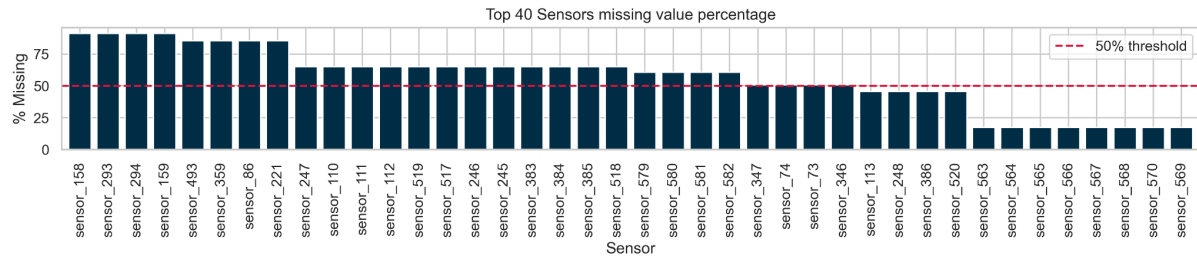


Figure 1. Top 40 sensors by missing value percentage, showing clear group above 50% threshold

Table 2 summarises the preprocessing outcomes.

Step	Before	After
Raw features	590	—
After constant removal	—	~585
After >50% missing removal	—	~480
Post-imputation and zero-var removal	—	446
Pass samples	1,463 (93.4%)	unchanged
Fail samples	104 (6.6%)	unchanged

Table 2. Summary of preprocessing steps and feature counts.

4.2 Exploratory Data Analysis (EDA)

EDA was conducted using statistical summaries and visualisations. The feature variance histogram shows a strong right skew before scaling. Most sensors have low variance, while only a few sensors show very high variance and dominate the raw feature space.

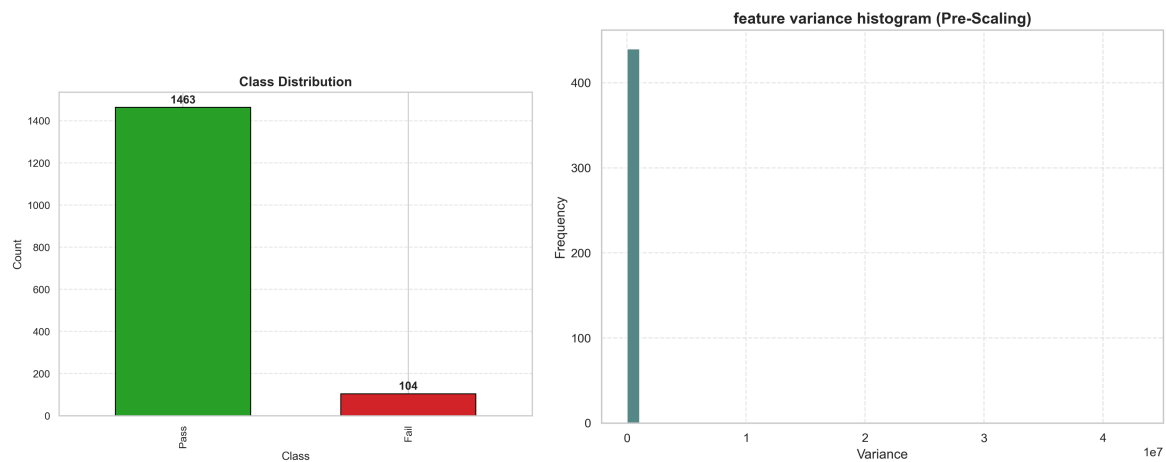


Figure 2. Class distribution and feature variance histogram before scaling

The top 30 sensors by standard deviation were plotted to identify the most variable measurements.

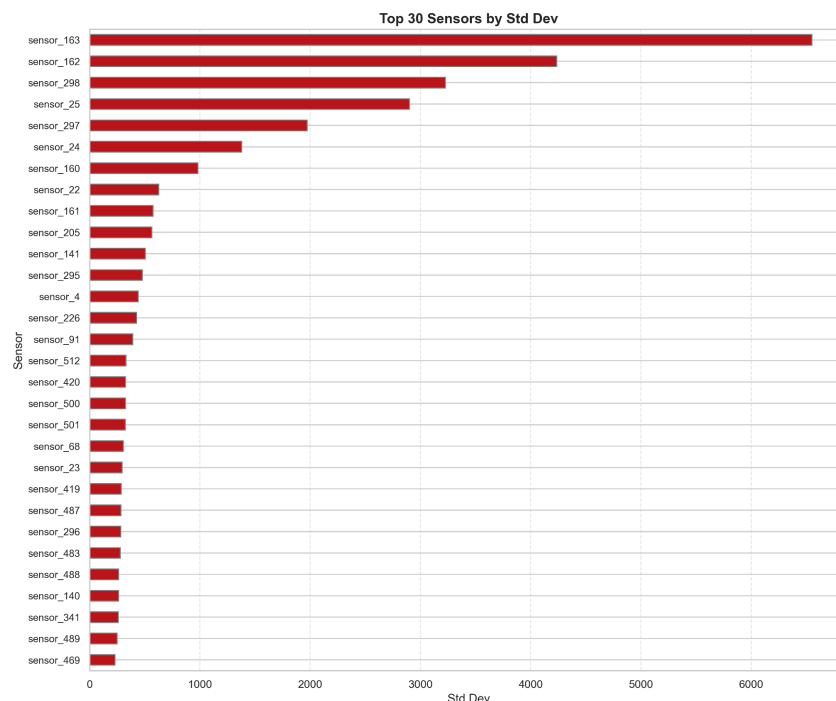


Figure 3. The top 30 sensors by standard deviation

A correlation heatmap of the top 20 highest-variance sensors were plotted and It revealed significant multicollinearity, with many sensor pairs showing correlations above 0.6

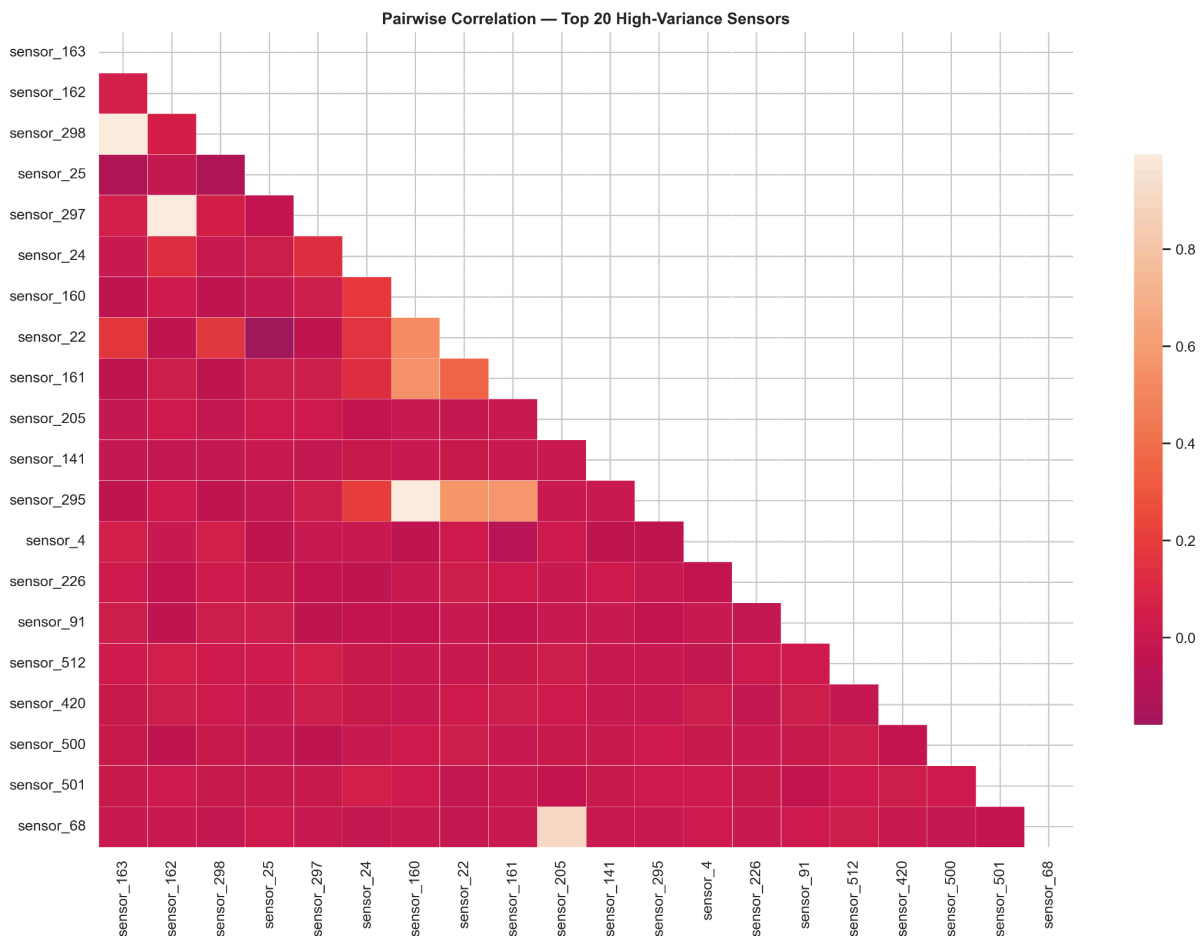


Figure 4. Correlation heatmap of top 20 high-variance sensors

This shows that the 446 features raw are quite redundant, and gives strong motivation for PCA, which is built precisely to extract orthogonal structure from correlated inputs. Welch's t-test was performed for each feature, and the top 8 features that discriminate Pass from Fail were visualised as box plots. This is the right place for applying the Welch variant since pass and fail sample sizes are very unequal and variances are different between classes.

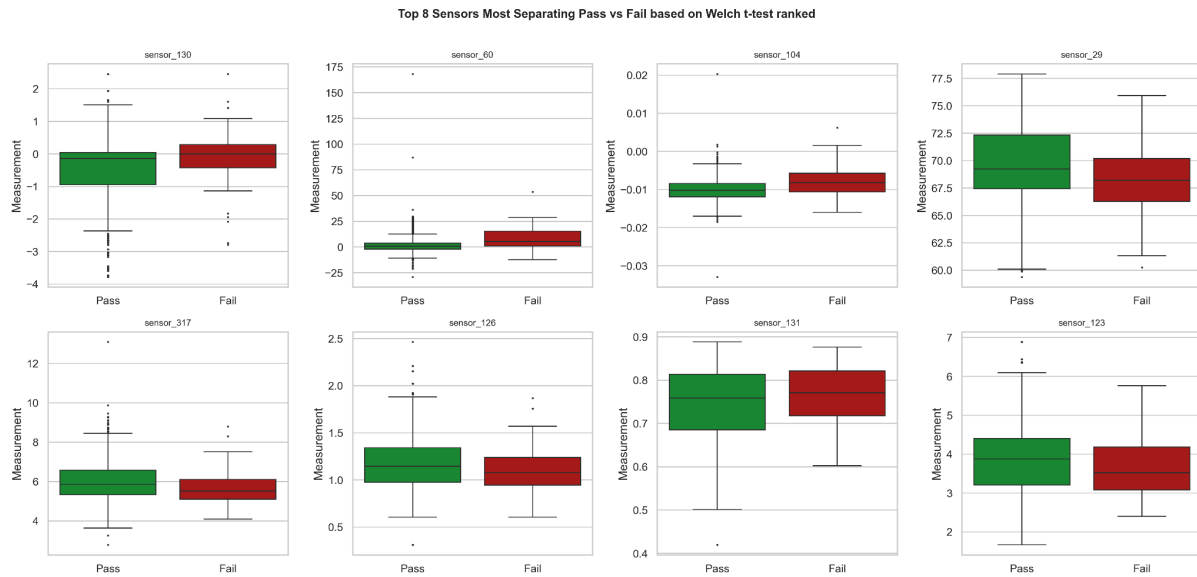


Figure 5. Top 8 sensors most separating Pass vs Fail (Welch t-test ranked)

The box plots showed that several sensors have meaningfully different distributions between outcomes, which encourages the expectation that a classifier can learn from them.

4.3 Outlier Analysis

Two complementary outlier detection methods were applied to the scaled features:

Z-Score method (threshold $|z| > 3$):

The absolute Z-score matrix was calculated across all scaled features. Almost every sample had at least one sensor reading beyond three standard deviations, this was expected in a high-dimensional sensor array with 446 features and a normal distribution, it becomes very likely that at least one will appear as an extreme value. When comparing classes, the average number of outlier sensors per sample was computed separately for pass and fail cases. The results from the z-score suggest that failed samples tend to show slightly more extreme sensor readings compared to passing samples.

IQR method (fence = $Q1 - 1.5 \times IQR$, $Q3 + 1.5 \times IQR$):

The IQR method supported the Z-score results, as most samples had several features falling outside the fence limits. After the analysis it is decided to retain the outliers. The rationale is

threefold. First, with 446 features, practically every sample would be flagged as an outlier on at least one dimension; removing them would destroy the dataset. Second, in fault detection, extreme sensor readings are precisely the signal we want to capture and removing them will remove fail samples too. Thirdly, the classifiers used here (SVM with RBF kernel, Decision Tree) are relatively robust to individual extreme values. StandardScaler was used to limit the influence of very large values on distance calculations.



Figure 6. Outlier analysis using Z-Score and IQR methods

4.4 Dimensionality Reduction : PCA

Principal Component Analysis (PCA) is an unsupervised linear dimensionality reduction method that transforms data from a higher-dimensional space of correlated features into a smaller dimension of orthogonal (or uncorrelated) variables, known as Principal components(PCs). Each PC is a linear combination of the original features pointing in the direction of maximum remaining variance. While the first PC captures the largest variance, the second captures the largest remaining variance orthogonal to (perpendicular to each other) the first principal component, and so on.

SECOM dataset can greatly benefit from PCA because:

- (a) the heatmap shows that many sensors are correlated, therefore effective dimensionality is much lower than 446.
- (b) the noisy signals in individual sensors may combine across components such that their reduced-dimension representation is much cleaner

(c) distance-based classifiers like k-NN and SVM are more suitable in lower-dimensional spaces due to the 'curse of dimensionality' consideration.

PCA Component Selection

Full PCA was applied to all 446 scaled features to examine how variance is distributed across components. Table 3 summarizes the number of components needed to retain different amounts of total variance, Full PCA revealed the following component thresholds:

Variance Threshold	Components Required	% of 446 Features
90%	127	~29%
95%	162	~36.3%
99%	216	~48.4%

Table 3. Components required at different variance thresholds

Figure 7 shows two complementary views of the variance distribution. The scree plot displays the variance explained by the first 60 principal components. There is a sharp drop from the first couple of components, followed by a long flat area where each additional component contributes very little. The right panel shows cumulative explained variance as the number of components increases.

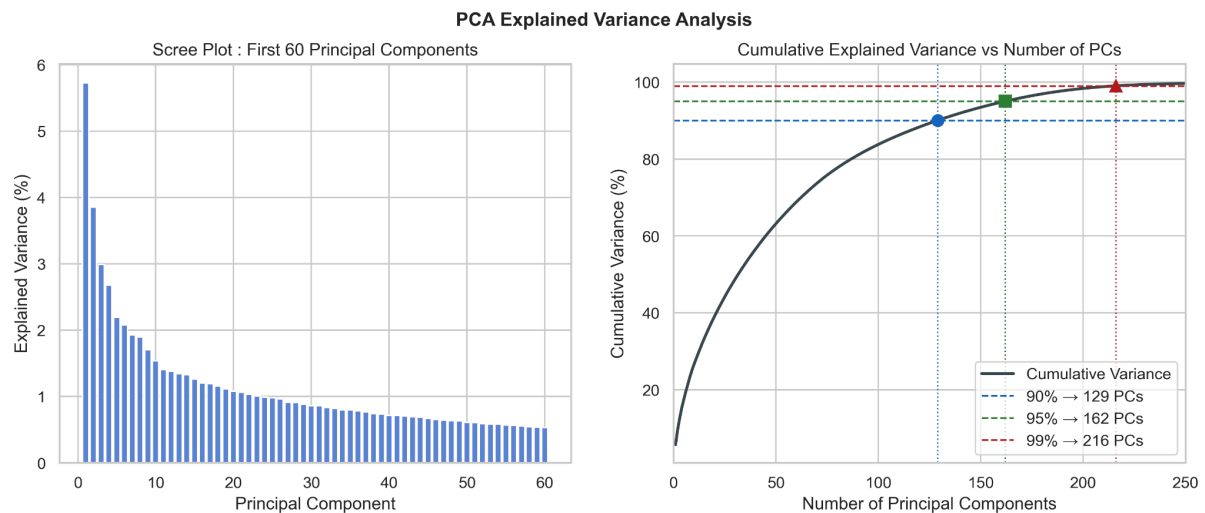


Figure 7. Scree plot and cumulative explained variance

2 components: 2-component PCA captures up to ~10–15% of the total variance, shown in Figure 7. This is helpful for providing an informative 2D scatter plot but keeps way too little information to make any accurate classification happen. If we only used 2 principal components to train classifiers you would throw away most of the signal separating pass from fail.

5 components: 20-25% of variance would maybe be retained but still less than 1/4th of the available information. This is still not good enough for a classification task, which needs every bit of discriminative power.

10 components: 10 components may explain ~35%–40% variable, better but still most of the useful variance not captured. There exists a significant performance gap of 10 components vs full 95%

95% threshold (selected): The variance threshold of 95% was a good fit. It performs a drastic reduction of over 450 dimensions down to only roughly 120 components, all orthogonal and containing almost no meaningful information. Figure 6: The screen plot confirms the elbow on explained variance per components that occurs rapidly after all but the first few PCs and then extends into a long tail. The 95% cut-off rests comfortably within this plateau, ensuring that no significant variance components are discarded.

99% threshold: Retaining 99% of variance would require around 200 components which is nearly half the original feature count. The additional 4% of variance beyond 95% comes from the extreme tail of the scree plot, where individual components contribute almost nothing or very less. This would increase computational cost and risk overfitting to noise without providing meaningful performance gains.

Figure 8 shows the projection of this data in two dimensions on each pair of those first two principal components (PC1 & PC2). PC1 accounts for about 6.7% variance and PC2 explains around 5.7%. This scatter plot indicates that pass (green) and fail (red) samples greatly overlap in this low-dimension space. This indeed confirms that the problem is not linearly separable even in 2D as there is no clear linear boundary between the two classes. The visual overlap provides motivation for why classification must be done in the full 95% PCA space (~120 components); structure that separates the two classes exists in many more dimensions than just the first two.

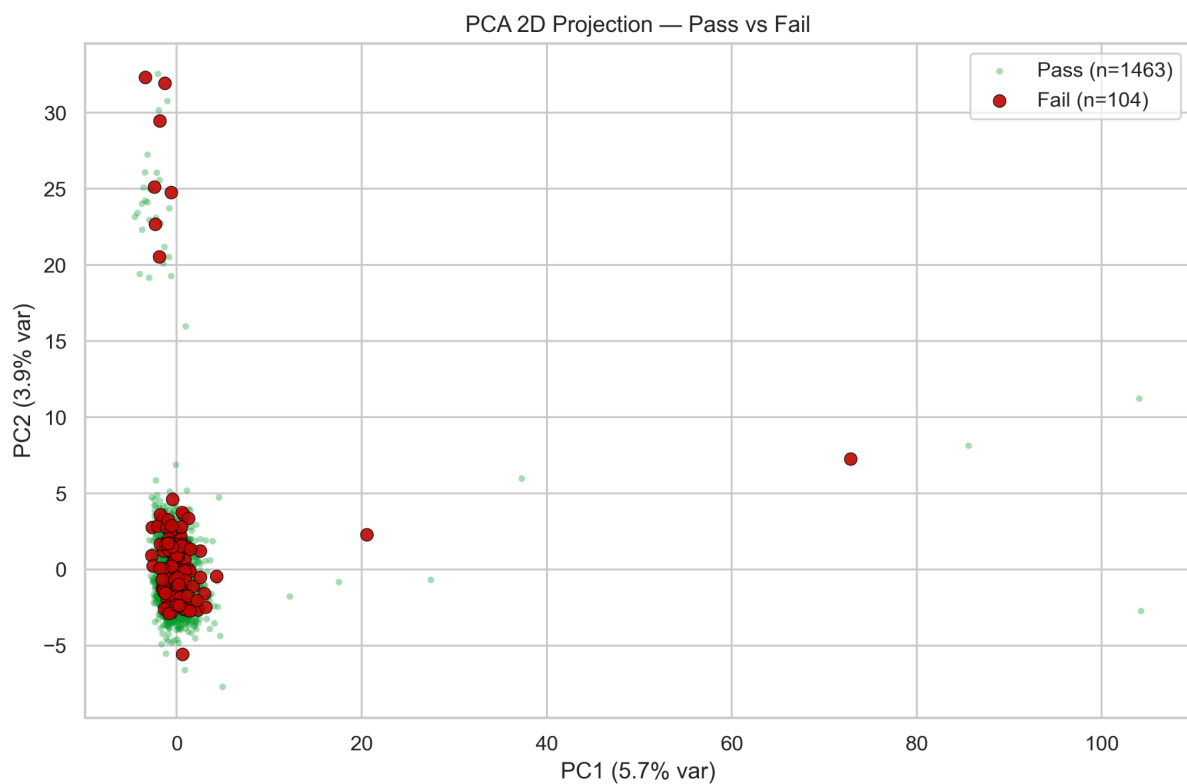


Figure 8. PCA 2D projection showing Pass vs Fail overlap

4.5 Clustering

What is K-Means Clustering?

K-Means Clustering is a clustering algorithm which works using partition-based data into k clusters. This means each sample is assigned to the closest centroid and then iteratively updates these centroids until it reaches an equilibrium point. The aim is to minimise the summed intracluster variance, also referred to as inertia. The key parameters include k (number of clusters), `random_state` (to make sure that the initial positions of both centroids are the same) and `n_init = 10`, since the algorithm run multiple times with different placements of the centroids to recover from local minima so we take into consideration only best result among ten runs with varying locations for finding minimum inertia.

What is Hierarchical Clustering?

Hierarchical clustering builds a dendrogram without requiring k initially. Ward linkage is used here to merge the pair of clusters that produces the smallest increase in total within-cluster variance at each step. It tends to produce compact, roughly equal-sized clusters. The dendrogram is particularly valuable here because it allows us to visually check how many clusters actually exist by searching for large vertical jumps in merge levels.

Why is clustering important in this case?

The SECOM dataset has no predefined process groups. Clustering explores whether the sensor data has any latent structure that might correlate with failure rates. If one cluster shows a much higher failure percentage than others, this finding is immediately actionable for engineers, even without a supervised model.

Parameter selection and results:

The elbow plot of inertia vs k showed a gradual bend rather than a sharp elbow, which is common in high-dimensional data. Silhouette score was chosen as the primary metric for optimising for the number of clusters. The result value lies between -1 to +1, where a high

value means that clusters are well-defined. In simple terms , it evaluates the proximity of each sample to its own cluster against other clusters around it.

k	Inertia	Silhouette Score
2	highest	best (selected)
3	lower	second
4–10	decreasing	declining or flat

Table 4. K-Means parameter sweep results

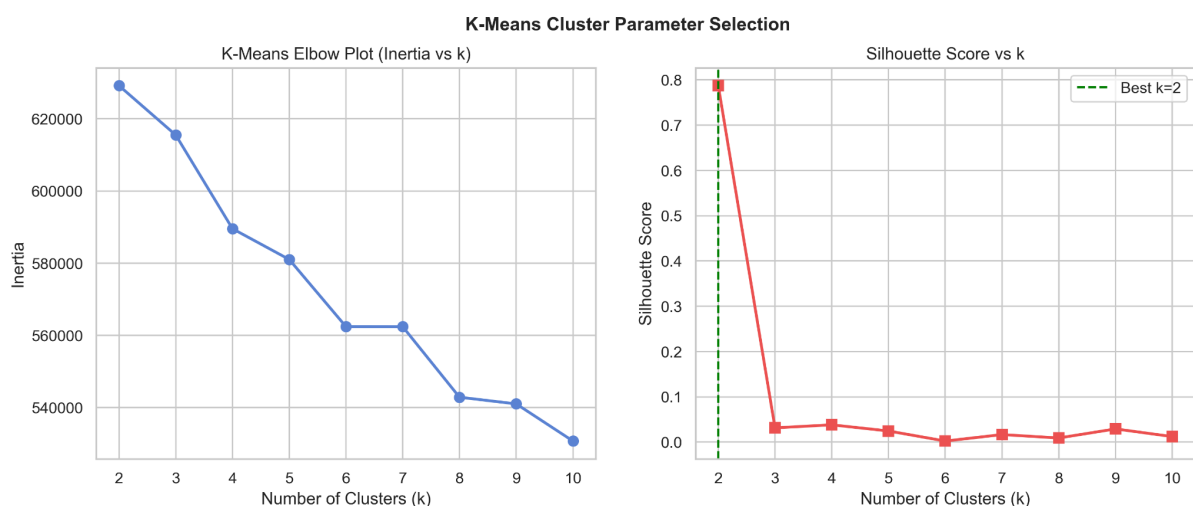


Figure 9. K-Means elbow plot and silhouette score analysis

The highest silhouette score was found for k=2, which is thus the optimal partition. With a clear separation between the two clusters:

- Cluster 0 : large cluster, low fail rate near dataset average (6.6%).
- Cluster 1: A smaller cluster with an exceptionally high fail rate, which means a process condition or equipment state that is found to be producing more defects (25% fail rate).

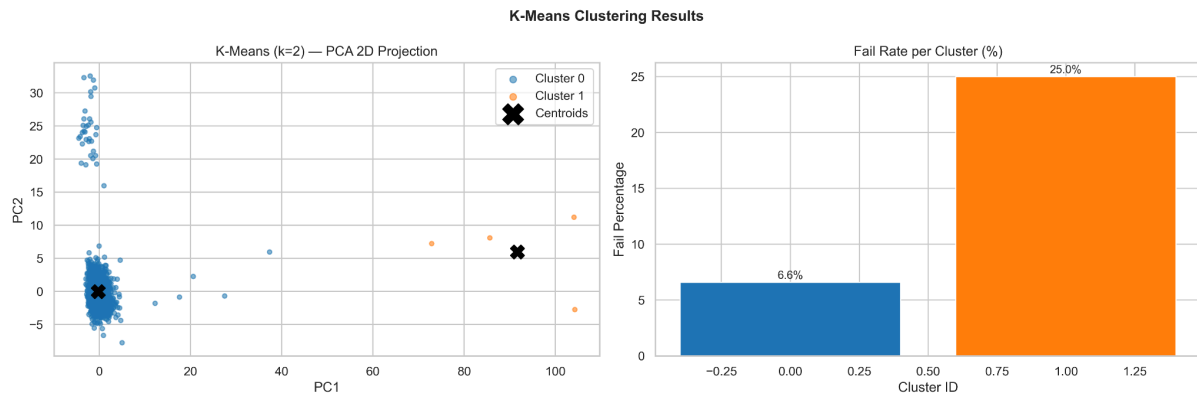


Figure 10. K-Means clustering results and fail rate per cluster

In either case, the Ward dendrogram on the 250-sample subsample gave independent support for a two- or three-cluster structure i.e. two /three large branches before the last merge were previously available. The agreement of the $k=2$ finding between K-Means and hierarchical clustering builds confidence to this result.

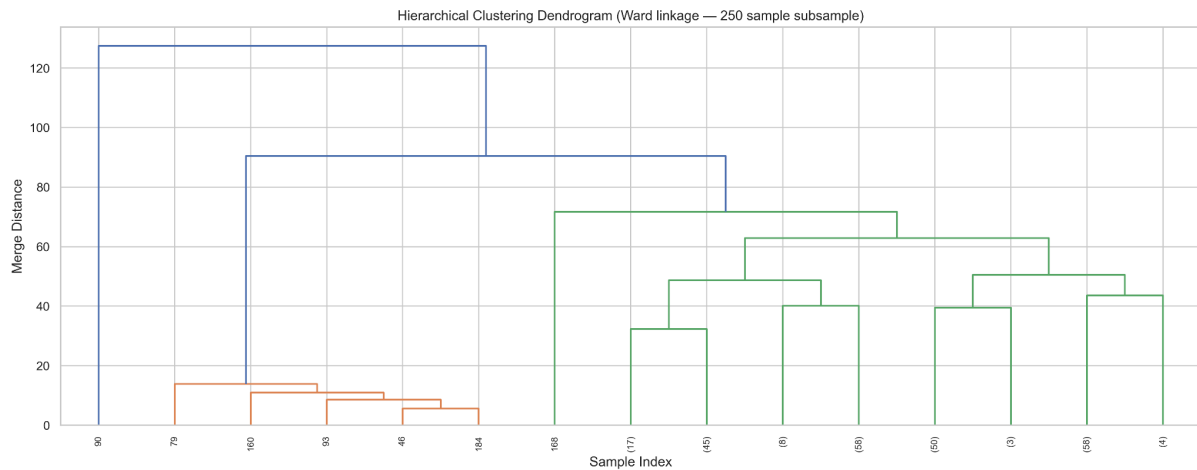


Figure 11. Hierarchical clustering dendrogram (Ward linkage)

4.6 Classification

The dataset was split into 80% training and 20% testing data, with stratification applied to the target variable so that the failure rate (6.6%) was preserved in both sets. SMOTE is applied to the training portion only creating a balanced training set containing the same number of pass and fail. Nothing was done on the test set.

k-Nearest Neighbours (k-NN): k-nearest neighbors (k-NN) is a supervised learning algorithm to classify a sample by examining its k nearest neighbors which class occurs most frequently. The key parameter is k, small k makes the model sensitive to noise (overfitting), while large k over-smooths the decision boundary (underfitting). GridSearchCV over odd k values from 3 to 29 with 5-fold CV optimising recall on the fail class selected the best k. CV recall scores across most k values were near 0.99 on the SMOTE-balanced training data, suggesting the synthetic samples were highly separable. Test recall for the tuned model was lower, which is around 0.67 and is discussed in the limitations section.

In the k-NN algorithm, a sample is classified to the majority of its k nearest neighbours in feature space. The main parameter here is k, a small k means overfitting (the model is sensitive to the noise) and a larger k leads to underfitting (over-smoothing of the decision boundary). The best k was selected from and over odd k values of 3 to 29 with GridSearchCV with 5-fold CV optimising recall on the fail class: here it is possible to see that CV recall scores were close to almost a perfect 0.99 across most, indicating the synthetic samples were very separable being SMOTE-balanced training data. The greater the number of false negatives we have, the lower our recall will be (for tuned models test recall was about 0.67, more on that in the limitations section).

Decision Tree: Decision Trees iteratively split the data on different feature values creating smaller and purer groups based on Gini or entropy. GridSearchCV was used to find the optimal values for tree depth (3 and above), minimum samples per leaf (1 ~ 10) and splitting criterion. Since the dataset was imbalanced it was used SMOTE, with class_weight='balanced', so that to the model to focus more on minority classes. The low depth of a tree used in the final model is necessary to prevent overfitting and simply memorizing the data.

Support Vector Machine (SVM): A support vector machine (SVM) with a radial basis function (RBF) kernel is an algorithm that will take the data and map it into a higher-dimensional space where the algorithm can find the clearest boundary between classes with maximum margin. C regulates the trade-off between a low error classifier and a wide margin while gamma determines how much influence one training example

has.RandomizedSearchCV was used to quickly test 30 combinations of C, gamma (on log scale).The chosen model uses an RBF kernel with optimal values of C and gamma.

4.7 Model Evaluation

What is accuracy?

The Accuracy measures the percentage of correct predictions made by the model. But, accuracy could be incorrect to use for imbalanced problems like SECOM. For instance if the model predicts every sample as “Pass”, it would still classify correctly 93.4% of the time while totally failing to catch any true failures. Because of this, accuracy was included only for reference and was not treated as the main evaluation metric.

What is a confusion matrix?

A confusion matrix is a 2×2 table that determines the performance of a model through counting True Positives (TP: correctly predicted failures), False Positives (FP: passing units predicted as failures), True Negatives (TN: correctly predicted passing units) and False Negatives (FN: failed units predicted as passing).

For example, in a manufacturing context, False Negatives are the most serious errors since defective items could end up with customers. False Positives are less of a concern as they just wrongly reject good units. Due to this recall, the number of actual failures that are correctly detected is considered as the most critical metric in this problem.

Metrics used: 6 evaluation measures are used to evaluate the models, namely: Accuracy, Precision for fail class, Recall for fail class, F1-score for fail class, Macro F1-score and AUC-PR (Area Under the Curve of the Precision–Recall) curve.

Model Comparison Table:						
Model	Accuracy	Precision (Fail)	Recall (Fail)	F1 (Fail)	F1 (Macro)	AUC-PR
k-NN	0.4522	0.0782	0.6667	0.1400	0.3691	0.0744
Decision Tree	0.8185	0.1087	0.2381	0.1493	0.5238	0.0768
SVM	0.9331	0.5000	0.0476	0.0870	0.5261	0.0875

Table 4. Test set performance for all three tuned classifiers

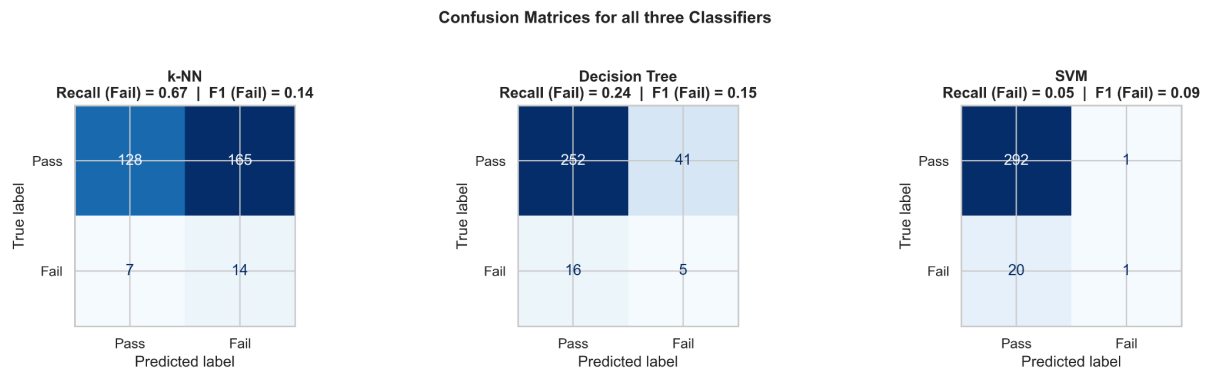


Figure 12. Confusion matrices for all three classifiers

Overall, the SVM showed the best overall performance of all tested models, with highest achieved macro F1-score and precision. Decision Tree also scored well and, additionally, is probably the easier option to interpret since you can clearly see what sensor values and thresholds are affected, add assessment of decision boundary.

That k-NN model has the highest recall, detecting more true failures and achieving the highest recall. But on the contrary, its lower accuracy means that it also gave more false positives by falsely recognizing failures for some passing samples.

Model	Status	Recall (Fail)	F1 (Fail)	F1 (Macro)
k-NN	Original	0.2919	0.125	0.7143
	Tuned	0.6667	0.14	0.3691
Decision Tree	Original	0.4911	0.1524	0.381
	Tuned	0.2381	0.1493	0.5238
SVM	Original	0.5235	0.0833	0.0476
	Tuned	0.0476	0.087	0.5261

Figure 13. Comparison of fine tuned and original models

Comparing the models before and after tuning showed that all baseline models using tuned parameters outperforms default parameters in terms of macro F1-score. After tuning the value of k using cross-validation, it was especially noticeable in the case of k-NN that as soon

as we chosen the best value for k, recall improved dramatically. The same was true for the SVM model, after tuning we can see that a very large C value was employed, enough larger than the default value of 1.0 to allow this noisy dataset to create entire decision boundaries using hard margins.

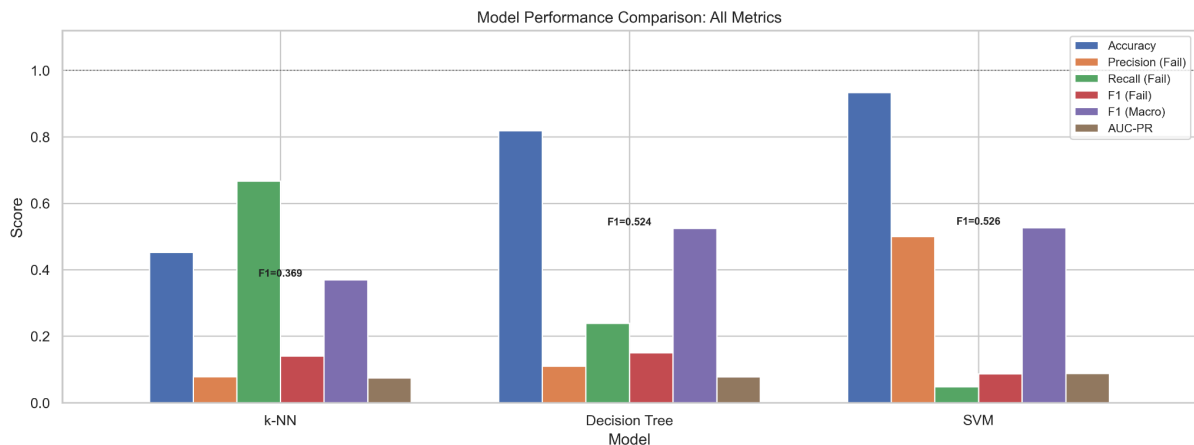


Figure 14. Model performance comparison across all metrics T

comparison with the three classifiers over six evaluation metrics in a grouped bar chart was shown at Figure 14. All three models had accuracy scores between 0.45 and 0.93, with SVM performing the best. However, accuracy can be misleading for the SECOM Dataset because the dataset is highly imbalanced. A model could achieve high accuracy by simply predicting every sample as “pass” while failing to detect actual failures.

In this situation, more relevant metrics would then be the precision and recall on the fail class. K-NN got the highest recall where more true failures were detected. However, k-NN showed low precision, which means it produced many false rates. The Decision Tree shows more balanced performance. The SVM achieved better balance between precision and recall by detecting failures while keeping false rate relatively low. The SVM achieved the highest F1-score as well as Macro F1-score, which indicates that it performed best overall among the three models.

As prediction in a highly imbalanced manufacturing dataset is difficult, hence the AUC-PR for all models was somewhat low. Overall, this chart illustrates that even if two models seem to be similar in terms of accuracy, they can actually perform quite differently when the main

goal is detecting failure. So, metrics like recall and F1-score are way more important than accuracy for performance evaluation of manufacturing defect detection systems.

5. Discussion

This section provides an analysis of the pipeline results, explaining why certain choices were made, what the models are actually doing, and where the approach could be improved.

5.1 Critical Analysis of Results

The behaviour of the three classifiers is different when they are tested on the test set. Although the seemingly high accuracy is found in all models (k-NN: 45.22%, Decision Tree: 81.85%, SVM: 93.31%) other than K-nn. The class imbalance found in the data results in misleading accuracy since a class label of “pass” already gives around 93% accuracy. Consequently, only inaccurate models are not valid indicators of model performance. The k-NN model has a fairly high recall (0.67), but very low precision (0.08), meaning that it identifies the majority of failures, but gives many false alarms. However, SVM has a more balanced and reliable performance with higher precision (0.50) and best macro F1-score (≈ 0.53), which is a better overall trade-off between classes with lower recall. The Decision Tree falls somewhere between the extremes, and has the added benefit of being easily explained to the engineer by the thresholds of the sensors which determine the decision rules. In general, the SVM model is the strongest model for this data set, k-NN has a higher rate of false positives, and the Decision Tree is not optimized but interpretable.

5.2 SMOTE and Optimism Bias

SMOTE is a very strong technique but one of the key points to consider in this project is that applying SMOTE prior to splitting the indices for cross validation can give a discrepancy between the cross validation performance and test performance. In 5-fold CV on SMOTE-balanced training data, all models performed much better: k-NN reached an F1(Macro) of 0.66, Decision Tree of 0.88 and SVM almost perfect with a score of 0.99 While on the real test set, those fell to 0.37, 0.52 and 0.53 respectively In particular, SVM falls some 47 points from 0.99 to 0.53. This is because SMOTE generates new minority data points based on an interpolation between two real data points, so that when we apply it

globally prior to the CV splitting, it is possible for synthetic data points to be the same real data point in both training and validation folds. The model essentially tries to see how it can fool itself with small perturbations of the data on which it was trained, and provides false, inflated estimates. Let's see how it goes, the test results tell: k-NN finds 66.67% of failures with 7.82 % accuracy (92 % false alarms), Decision Tree finds only 23.81%, and SVM finds only 4.76%, and the level of misses is 95%! But SMOTE should be done within each CV fold and pipeline wrappers should be used to make sure that validation data is truly unseen and performance estimates are honest in the future.

5-Fold CV F1(macro) on balanced training data:

k-NN	F1=0.6606 ±0.0298
Decision Tree	F1=0.8784 ±0.0153
SVM	F1=0.9944 ±0.0035

5.3 Limitations

The current approach has a number of important drawbacks that impact how realistic and interpretable the results are. Firstly, it does not take into account the fact that the data is based on time. The data set includes timestamps for each sample, but all of the observations were at a single moment in time and were assumed to be independent observations. An actual manufacturing environment, however, would be time-dependent, with the behaviour of the sensors changing over time as a result of wear and drift and thus might show better results when using a temporal approach to detecting changes in the way the failure particles evolve. Secondly, complexity of interpretation is decreased. It works best with data with high dimensions, but it turns the 446 data from the sensors into a combined data. The model is not very useful for practical diagnostics or maintenance since it is difficult for the engineers to find out which sensors among the many, are trying to bring on an anticipated failure.

6. Conclusion

This project applied a full machine learning pipeline to the SECOM Dataset. The data started with 590 sensor features, many of which had missing values and noise. After cleaning, 446 useful features were kept, and PCA is used as the dimensionality reduction technique used here to reduce them further while still preserving about 95% of the important information. Unsupervised analysis showed that the data naturally formed two clusters, and one of these clusters had a much higher failure rate. This suggests there may be hidden machine or process states behind the sensor readings. Three models are trained and then fine tuned: k-NN (focusing on recall), Decision Tree (focusing on interpretability and balanced performance), SVM with RBF kernel (best overall balance). In general, the SVM had the best performance among all models based on having roughly even precision and recall together with the most favorable macro F1-score value as well as AUC-PR pairs.

The evaluation process did identify one notable issue, however, applying SMOTE before cross-validation caused data leakage, making validation results overly optimistic. This means the true performance on test data was lower than expected. For future work, SMOTE should be applied correctly inside each cross-validation fold. Other improvements could include using the time information in the data, and exploring cost-sensitive learning instead of oversampling.

7. References

- [1] A. J. Michael McCann, "SECOM." UCI Machine Learning Repository, 2008. doi: 10.24432/C54305.
- [2] I. T. Jolliffe, *Principal Component Analysis*, 2nd edition. New York, NY, USA: Springer Science & Business Media.
- [3] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, Jun. 2002, doi: 10.1613/jair.953.
- [4] G. Douzas, F. Bacao, and F. Last, "Improving imbalanced learning through a heuristic oversampling method based on k-means and SMOTE," *Inf. Sci.*, vol. 465, pp. 1–20, Oct. 2018, doi: 10.1016/j.ins.2018.06.056.
- [5] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, Sep. 1995, doi: 10.1007/BF00994018.
- [6] Chih-Chung Chang, Chih-Jen Lin, and Chih-Wei Hsu, "A Practical Guide to Support Vector Classification," Department of Computer Science and Information Engineering, National Taiwan University, Taipei, Taiwan, 2003. [Online]. Available: <https://www.csie.ntu.edu.tw/~cjlin/papers/guide/guide.pdf>

8. Appendix

1. Data Understanding & Preprocessing

```
1 import warnings;
2 warnings.filterwarnings('ignore')
3 import numpy as np
4 import pandas as pd
5 import matplotlib.pyplot as plt
6 import seaborn as sns
7 import os, time
8 from scipy import stats
9 from scipy.stats import ttest_ind
10 from scipy.cluster.hierarchy import dendrogram, linkage
11
12 from sklearn.preprocessing import StandardScaler
13 from sklearn.decomposition import PCA
14 from sklearn.cluster import KMeans
15 from sklearn.neighbors import KNeighborsClassifier
16 from sklearn.tree import DecisionTreeClassifier, plot_tree
17 from sklearn.svm import SVC
18 from sklearn.model_selection import train_test_split, cross_val_score
19 from sklearn.metrics import (
20     accuracy_score, precision_score, recall_score, f1_score,
21     classification_report, ConfusionMatrixDisplay, silhouette_score,
22     average_precision_score
23 )
24 from imblearn.over_sampling import SMOTE
25 from sklearn.tree import plot_tree
26 from sklearn.model_selection import GridSearchCV
```

```

27 from sklearn.model_selection import RandomizedSearchCV
28 from scipy.stats import loguniform
29
30 np.random.seed(42)
31 sns.set_theme(style='whitegrid', palette='deep')
32 pass_clr, fail_clr = '#069e2d', '#bf0603'

```

```

1  #Helper Functions
2
3  def load_secom_data(data_path: str, labels_path: str)-> tuple:
4      """
5      To load SECOM sensor data and labels
6      return (DataFrame, ndarray).
7      """
8      sensor_df = pd.read_csv(data_path, sep='\s+', header=None, na_values='NaN')
9      sensor_df.columns = [f'sensor_{i+1}' for i in range(sensor_df.shape[1])]
10     label_df= pd.read_csv(labels_path, sep='\s+', header=None, names=['outcome',
11 'timestamp'])
12     target = (label_df['outcome'] == 1).astype(int).values # 1=Fail, 0=Pass
13     return sensor_df, target
14
15
16 def preprocess_data(secom_data_raw: pd.DataFrame, missing_thresh: float = 0.50)
17 -> pd.DataFrame:
18     """Drop constant/high-missing cols, median-impute, remove post-impute
19 zero-variance cols."""
20     cleaned = secom_data_raw.copy()
21     cleaned.drop(columns=cleaned.columns[cleaned.nunique(dropna=False) <= 1],

```

```

22 inplace=True)
23         cleaned.drop(columns=cleaned.columns[cleaned.isnull().mean() >
24 missing_thresh], inplace=True)
25     cleaned.fillna(cleaned.median(), inplace=True)
26     cleaned.drop(columns=cleaned.columns[cleaned.std() == 0],inplace=True)
27     return cleaned
28
29
30 def scale_features(cleaned_sensor_data: pd.DataFrame) -> tuple:
31     """
32     Tofit StandardScaler
33     """
34     fitted_scaler = StandardScaler()
35     scaled_array = fitted_scaler.fit_transform(cleaned_sensor_data)
36     return scaled_array, fitted_scaler
37
38
39 def get_top_discriminating_features(secom_data_raw: pd.DataFrame, target:
40 np.ndarray,top_n: int = 8) -> list:
41     """
42     To return top_n feature names most separating Pass/Fail by Welch t-test statistic.
43     return List
44     """
45     t_scores = {
46         col: abs(ttest_ind(secom_data_raw[col][target == 0],
47                             secom_data_raw[col][target == 1],
48                             equal_var=False).statistic)
49         for col in secom_data_raw.columns

```

```

50     }
51     return sorted(t_scores, key=t_scores.get, reverse=True)[:top_n]
52
53
54 def evaluate_classifier(name: str, y_true: np.ndarray, y_pred: np.ndarray) -> dict:
55     """To return accuracy, precision, recall, F1-fail F1-macro for corresponding
56 classifier."""
57     return {
58         'Model':      name,
59         'Accuracy':    round(accuracy_score(y_true, y_pred), 4),
60         'Precision (Fail)': round(precision_score(y_true, y_pred, zero_division=0), 4),
61         'Recall (Fail)': round(recall_score(y_true, y_pred, zero_division=0), 4),
62         'F1 (Fail)':    round(f1_score(y_true, y_pred, zero_division=0), 4),
63         'F1 (Macro)':    round(f1_score(y_true, y_pred, average='macro',
64 zero_division=0), 4),
65         'AUC-PR':      round(average_precision_score(y_true, y_pred), 4),
66     }

```

```

1  #load data
2      dir_path = "/Users/arjun/Documents/data
3 mining/Data-mining-assignment-2/secom"
4  sensor_df, target = load_secom_data(
5      os.path.join(dir_path, 'secom.data'),
6      os.path.join(dir_path, 'secom_labels.data')
7  )
8
9  print(f"Raw shape: {sensor_df.shape}")

```

```
print(f"Pass: {(target==0).sum()} | Fail: {(target==1).sum()} | Fail rate:
{target.mean()*100:.1f}%")
```

```
1 #missing value detection
2 missing_percentage = sensor_df.isnull().mean() * 100
3 print(f"Sensors with any missing values : {(missing_percentage > 0).sum()}")
4 print(f"Sensors with more than 50% missing: {(missing_percentage > 50).sum()}")
5 print(f"Total missing data(cells): {sensor_df.isnull().sum().sum():,}")
6
7 fig,ax = plt.subplots(figsize=(13, 3))
8     missing_percentage.sort_values(ascending=False).head(40).plot.bar(ax=ax,
9 color='#003049', width=0.8)
10 ax.axhline(50, color='crimson', ls='--', lw=1.5, label='50% threshold')
11 ax.set(title='Top 40 Sensors missing value percentage', ylabel='% Missing',
12 xlabel='Sensor')
13 ax.legend()
14 plt.tight_layout();
    plt.savefig('sensor_missing_values.png', dpi=300, bbox_inches='tight')
    plt.show()
```

```
1 #Preprocessing and scaling
2 cleaned_data = preprocess_data(sensor_df, missing_thresh=0.50)
3 features_scaled, std_scaler = scale_features(cleaned_data)
4
5 print(f"Sensors before preprocessing : {sensor_df.shape[1]}")
6 print(f"Sensors after preprocessing : {cleaned_data.shape[1]}")
```

7	<code>print(f"Samples : {features_scaled.shape[0]}")</code>
8	<code>print(f"Remaining missing cells : {np.isnan(features_scaled).sum()}")</code>

2. Exploratory Data Analysis

1	<code>#Class imbalance chart</code>
2	<code>fig, axes = plt.subplots(figsize=(8, 6))</code>
3	<code>class_counts = pd.Series(target).map({0: 'Pass', 1: 'Fail'}).value_counts()</code>
4	<code>class_counts.plot.bar(</code>
5	<code>ax=axes,</code>
6	<code>color=['#2CA02C', '#D62728'],</code>
7	<code>edgecolor='black',</code>
8	<code>width=0.6</code>
9	<code>)</code>
10	
11	<code>for rect in axes.patches:</code>
12	<code>axes.text(rect.get_x() + rect.get_width()/2,rect.get_height() +</code>
13	<code>20,f"{int(rect.get_height())}",ha='center',fontsize=12,fontweight='bold')</code>
14	
15	<code>axes.set_title('Class Distribution', fontsize=14, fontweight='bold')</code>
16	<code>axes.set_xlabel('Class', fontsize=12)</code>
17	<code>axes.set_ylabel('Count', fontsize=12)</code>
18	<code>axes.tick_params(axis='both', labels=11)</code>
19	<code>axes.grid(axis='y', linestyle='--', alpha=0.4)</code>
20	<code>plt.tight_layout()</code>
21	<code>plt.savefig('class_distribution.png', dpi=300, bbox_inches='tight')</code>
	<code>plt.show()</code>

```

1 #feature variance histogram
2 fig, axes = plt.subplots(figsize=(8, 6))
3 feature_variances = cleaned_data.var().values
4 axes.hist(feature_variances,
5           bins=40,color='#588b8b',edgecolor='white')
6           axes.set_title('feature           variance           histogram
7 (Pre-Scaling)',fontsize=14,fontweight='bold')
8 axes.set_xlabel('Variance', fontsize=12)
9 axes.set_ylabel('Frequency', fontsize=12)
10 axes.tick_params(axis='both', labels=11)
11 axes.grid(axis='both', linestyle='--', alpha=0.4)
12 plt.tight_layout()
13 plt.savefig('feature_variance.png', dpi=300, bbox_inches='tight')
    plt.show()

```

```

1 #standard deviation of top 30 sensors
2 fig, ax = plt.subplots(figsize=(12, 10))
3 top_sensors = cleaned_data.std().nlargest(30).sort_values()
4 top_sensors.plot.barh( ax=ax,color='#ba181b',edgecolor='grey')
5 ax.set_title('Top 30 Sensors by Std Dev',fontsize=14,fontweight='bold')
6 ax.set_xlabel('Std Dev', fontsize=12)
7 ax.set_ylabel('Sensor', fontsize=12)
8 ax.tick_params(axis='y', labels=10)
9 ax.tick_params(axis='x', labels=11)
10 ax.margins(y=0.01)
11 ax.grid(axis='x', linestyle='--', alpha=0.4)
12 plt.tight_layout()

```

```

13 plt.savefig('top_30.png', dpi=300, bbox_inches='tight')
14 plt.show()

```

```

1 #correlation Heatmap fpr top 20 high-variance sensors
2 top20_sensors = cleaned_data.std().nlargest(20).index
3 corr_matrix =cleaned_data[top20_sensors].corr()
4 mask_upper= np.triu(np.ones((20, 20), dtype=bool))
5 fig, ax = plt.subplots(figsize=(14, 10))
6 sns.heatmap(corr_matrix, mask=mask_upper,cmap='rocket', center=0,
7             linewidths=0.2, annot=False,ax=ax,cbar_kws={'shrink': 0.7})
8 ax.set_title('Pairwise Correlation — Top 20 High-Variance Sensors',fontsize=12,
9 fontweight='bold')
10 plt.tight_layout();
11 plt.savefig('heat_map.png', dpi=300, bbox_inches='tight')
    plt.show()

```

```

1 #top-8 discriminative sensors
2 top_discriminating = get_top_discriminating_features(cleaned_data, target,
3 top_n=8)
4 fig, axes = plt.subplots(2, 4, figsize=(17, 8))
5 for ax, sensor_col in zip(axes.flatten(), top_discriminating):
6                                     plot_data =
7 pd.DataFrame({'Measurement':cleaned_data[sensor_col],'Outcome':np.where(target == 0,'Pass', 'Fail'))
8
9     sns.boxplot(x='Outcome', y='Measurement',data=plot_data,palette={'Pass':
10 pass_clr, 'Fail': fail_clr},flierprops=dict(marker='.', markersize=2), ax=ax)
    ax.set_title(sensor_col, fontsize=9)

```

```

11     ax.set_xlabel("")
12     plt.suptitle('Top 8 Sensors Most Separating Pass vs Fail based on Welch t-test
ranked',fontsize=12, fontweight='bold', y=1.01)
plt.tight_layout();
plt.savefig('box_plot.png', dpi=300, bbox_inches='tight')
plt.show()

```

3. Outlier Analysis

```

1  #Z-Score, to count how many features per sample exceed 3 standard deviations
2  abs_z_matrix = np.abs(stats.zscore(features_scaled))
3  z_outlier_count = (abs_z_matrix > 3).sum(axis=1)
4  print("Z-Score Analysis (threshold = 3):")
5  print(f" Samples with >= 1 outlier sensor : {(z_outlier_count > 0).sum()}")
6  print(f" ßMean outlier sensors per sample: {z_outlier_count.mean():.2f}")
7  print(f" Pass mean: {z_outlier_count[target==0].mean():.2f} ")
8  print(f" Fail mean: {z_outlier_count[target==1].mean():.2f} ")

```

```

1  # IQR
2      features_df_temp      =      pd.DataFrame(features_scaled,
3  columns=cleaned_data.columns)
4  q1_vals = features_df_temp.quantile(0.25)
5  q3_vals = features_df_temp.quantile(0.75)
6  iqr_range = q3_vals - q1_vals
7  lower_fence = q1_vals - 1.5 * iqr_range
8  upper_fence = q3_vals + 1.5 * iqr_range
9  iqr_outlier_count = ((features_df_temp < lower_fence) | (features_df_temp >
10 upper_fence)).sum(axis=1)
11  print("IQR Analysis")

```

```
print(f"samples with outliers : {(iqr_outlier_count > 0).sum()}")
print(f"Average outliers/sample: {iqr_outlier_count.mean():.2f}")
```

```
1 #outlier visualisation
2 fig, axes = plt.subplots(1, 3, figsize=(17, 4))
3 axes[0].hist(z_outlier_count, bins=40, color='#EF5350', edgecolor='white')
4 axes[0].set(title='Z-Score: Outlier Sensor Count per Sample', xlabel='Number of
5 sensors with |z|>3', ylabel='Samples')
6 #z-score count by class
7 bp = axes[1].boxplot(
8     [z_outlier_count[target == 0], z_outlier_count[target == 1]],
9     labels=['Pass', 'Fail'], patch_artist=True,
10    medianprops=dict(color='white', linewidth=2)
11 )
12 for patch, colour in zip(bp['boxes'], [pass_clr, fail_clr]):
13     patch.set_facecolor(colour)
14 axes[1].set(title='Outlier Count by Class (Z-Score)', ylabel='Outlier sensor count')
15 #IQR count distribution
16 axes[2].hist(iqr_outlier_count, bins=40, color='#FF8F00', edgecolor='white')
17 axes[2].set(title='IQR: Outlier Sensor Count per Sample', xlabel='Number of sensors
18 outside 1.5×IQR fence', ylabel='Samples')
19 plt.suptitle('Outlier Analysis — Z-Score and IQR Methods', fontsize=12,
20 fontweight='bold')
plt.savefig('Three_panel_outlier.png', dpi=300, bbox_inches='tight')
plt.tight_layout()
plt.show()
```

4. Dimensionality Reduction

```

1  #fit pca
2  pca_model = PCA(random_state=42).fit(features_scaled)
3  cumulative_variance = np.cumsum(pca_model.explained_variance_ratio_)
4  explained_variance_ratio = pca_model.explained_variance_ratio_
5  nc_90 = int(np.searchsorted(cumulative_variance, 0.90)) + 1
6  nc_95 = int(np.searchsorted(cumulative_variance, 0.95)) + 1
7  nc_99 = int(np.searchsorted(cumulative_variance, 0.99)) + 1
8  print(f"Components needed for 90% variance : {nc_90}")
9  print(f"Components needed for 95% variance : {nc_95}")
10 print(f"Components needed for 99% variance : {nc_99}")
11 print(f"\nFirst 10 individual explained variance ratios:")
12 for idx, val in enumerate(explained_variance_ratio[:10], 1):
13     print(f" PC{idx:>2}:{val*100:.2f}%")

```

```

1  fig, axes = plt.subplots(1, 2, figsize=(14, 5))
2
3  #scree plot(first 60 PCs)
4  axes[0].bar(range(1, 61), explained_variance_ratio[:60] * 100, color='#5C85D6',
5  width=0.8)
6  axes[0].set(title='Scree Plot : First 60 Principal Components',
7  xlabel='Principal Component', ylabel='Explained Variance (%)')
8
9  # Cumulative variance with thresholds
10 axes[1].plot(range(1, len(cumulative_variance)+1), cumulative_variance *
11 100,color='#37474F', lw=2, label='Cumulative Variance')
12 threshold_info = [(nc_90, 90, '#1565C0', 'o'), (nc_95, 95, '#2E7D32', 's'), (nc_99, 99,
13 '#B71C1C', '^')]
14 for nc, pct, col, mk in threshold_info:

```

```

15 axes[1].axhline(pct, color=col, ls='--', lw=1.2, label=f'{pct}% → {nc} PCs')
16 axes[1].axvline(nc, color=col, ls=':', lw=1.0)
17 axes[1].plot(nc, pct, marker=mk, color=col, ms=8, zorder=5)
18 axes[1].set(title='Cumulative Explained Variance vs Number of PCs',
19             xlabel='Number of Principal Components', ylabel='Cumulative Variance
20 (%)', xlim=(0, 250))
21 axes[1].legend()
22
plt.suptitle('PCA Explained Variance Analysis', fontsize=13, fontweight='bold')
plt.savefig('screepplot.png', dpi=300, bbox_inches='tight')
plt.tight_layout()
plt.show()

```

```

1 #PCA: 95% variance
2 pca_model = PCA(n_components=nc_95, random_state=42)
3 features_pca = pca_model.fit_transform(features_scaled)
4 pca_2d = PCA(n_components=2, random_state=42)
5 scores_2d = pca_2d.fit_transform(features_scaled)
6 scores_df = pd.DataFrame(scores_2d, columns=['PC1', 'PC2'])
7 print(scores_df.head())
8 print(f"\nshape      reduced:      {features_scaled.shape[1]}      sensors      =>
9 {features_pca.shape[1]} PCs")
10 print(f"Variance      retained      :
11 {pca_model.explained_variance_ratio_.sum()*100:.2f}%")
12
13 #scatterplot
14 fig, ax = plt.subplots(figsize=(9, 6))
15 for cls_val, cls_lbl, col, sz, alpha in [

```

```

16     (0, 'Pass', pass_clr, 12, 0.35), (1, 'Fail', fail_clr, 45, 0.90)]:
17     mask = target == cls_val
18     ax.scatter(scores_2d[mask, 0], scores_2d[mask, 1], c=col, label=f'{cls_lbl}
19 (n={mask.sum()}),
20     s=sz, alpha=alpha, edgecolors='black' if cls_val==1 else 'none', lw=0.4)
21 ax.set(title='PCA 2D Projection — Pass vs Fail',
22     xlabel=f'PC1 ({explained_variance_ratio[0]*100:.1f}% var)',
23     ylabel=f'PC2 ({explained_variance_ratio[1]*100:.1f}% var)')
24 ax.legend()

plt.tight_layout();

plt.savefig('pca.png', dpi=300, bbox_inches='tight')

plt.show()

```

5. Clustering

```

1 k_range = range(2, 11)
2 inertia = []
3 silhouette = []
4 for k in k_range:
5     model = KMeans(n_clusters=k, random_state=42, n_init=10)
6     labels = model.fit_predict(features_pca)
7
8     inertia.append(model.inertia_)
9     silhouette.append(silhouette_score(features_pca, labels, sample_size=600))
10
11 best_k = k_range[np.argmax(silhouette)]
12 optimal_k = best_k
13 k_search_range = k_range
14 inertia_vals = inertia
15 sil_vals = silhouette

```

```

16
17 results = pd.DataFrame({
18     "k": k_range,
19     "inertia": inertia,
20     "silhouette": silhouette
21 })
22
23 print(results.to_string(index=False))
24 print(f"\nBest k by silhouette: {best_k}")

```

```

1 #Elbow and Silhouette plots
2 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
3
4 axes[0].plot(list(k_search_range), inertia_vals, 'o-', color='#5C85D6', lw=2, ms=7)
5 axes[0].set(title='K-Means Elbow Plot (Inertia vs k)', xlabel='Number of Clusters (k)',
6 ylabel='Inertia')
7 axes[0].set_xticks(list(k_search_range))
8 axes[1].plot(list(k_search_range), sil_vals, 's-', color='#EF5350', lw=2, ms=7)
9 axes[1].axvline(optimal_k, color='green', ls='--', lw=1.5, label=f'Best k={optimal_k}')
10 axes[1].set(title='Silhouette Score vs k', xlabel='Number of Clusters (k)',
11 ylabel='Silhouette Score')
12 axes[1].set_xticks(list(k_search_range))
13 axes[1].legend()
14
15 plt.suptitle('K-Means Cluster Parameter Selection', fontsize=13, fontweight='bold')
16 plt.savefig('elbow_plot.png', dpi=300, bbox_inches='tight')
17
18 plt.tight_layout()
19
20 plt.show()

```

```

1  #k-means with the optimal k
2  means_model = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
3  km_assignments = means_model.fit_predict(features_pca)
4  final_sil = silhouette_score(features_pca, km_assignments, sample_size=600,
5  random_state=42)
6  print(f"Final K-Means,k={optimal_k} and Silhouette:{final_sil:.4f}")
7  fig, axes = plt.subplots(1, 2, figsize=(15, 5))
8
9  palette_km = sns.color_palette('tab10', optimal_k)
10 for c_idx in range(optimal_k):
11     cluster_mask = km_assignments == c_idx
12     axes[0].scatter(scores_2d[cluster_mask, 0], scores_2d[cluster_mask, 1],
13                     c=[palette_km[c_idx]], label=f'Cluster {c_idx}',alpha=0.55, s=15)
14
15     centroids_2d =
16     pca_2d.transform(pca_model.inverse_transform(means_model.cluster_centers_))
17     axes[0].scatter(centroids_2d[:, 0], centroids_2d[:, 1],marker='X', c='black', s=150,
18 zorder=10, label='Centroids')
19     axes[0].set(title=f'K-Means (k={optimal_k}) — PCA 2D Projection',xlabel='PC1',
20 ylabel='PC2')
21 axes[0].legend(markerscale=1.5)
22
23 #fail-rate per cluster
24 cluster_fail_pct = [
25     target[km_assignments == c_idx].mean() * 100
26     for c_idx in range(optimal_k)
27 ]

```

```

28
29 bar_colours = [palette_km[i] for i in range(optimal_k)]
30     axes[1].bar(range(optimal_k),      cluster_fail_pct,      color=bar_colours,
31 edgecolor='white')
32 for bar_idx, pct_val in enumerate(cluster_fail_pct):
33     axes[1].text(bar_idx, pct_val + 0.3, f'{pct_val:.1f}%', ha='center', fontsize=10)
34     axes[1].set(title='Fail Rate per Cluster (%)', xlabel='Cluster ID', ylabel='Fail
35 Percentage')
36 plt.suptitle('K-Means Clustering Results', fontsize=13, fontweight='bold')
37 plt.tight_layout()
38 plt.savefig('k_means.png', dpi=300, bbox_inches='tight')
39 plt.show()
40
    print("\nCluster Composition:")
    for c_idx in range(optimal_k):
        n_total = (km_assignments == c_idx).sum()
        n_fail = target[km_assignments == c_idx].sum()
        print(f" Cluster {c_idx}: {n_total:4d} samples | "
              f"Pass={n_total-n_fail} | Fail={n_fail} | Fail%={cluster_fail_pct[c_idx]:.1f}%")

```

```

1  #Hierarchical Clustering, sub sample 250
2  np.random.seed(42)
3  subsample_idx = np.random.choice(len(features_pca), size=250, replace=False)
4  X_sub_hier = features_pca[subsample_idx, :15] #subsampled, first 15 pcs
5  linkage_matrix = linkage(X_sub_hier, method='ward')
6
7  fig, ax = plt.subplots(figsize=(15, 6))
8  dendrogram(linkage_matrix, ax=ax, truncate_mode='level', p=5,

```

```

9         leaf_rotation=90, leaf_font_size=8,
10         color_threshold=0.7 * max(linkage_matrix[:, 2]))
11     ax.set(title='Hierarchical Clustering Dendrogram (Ward linkage — 250 sample
12 subsample)',
13           xlabel='Sample Index', ylabel='Merge Distance')
14     plt.tight_layout()
15     plt.savefig('ward_linkage_dendrogram.png', dpi=300, bbox_inches='tight')
16     plt.show()

```

6. Classification

```

1  #train / test split + SMOTE Balancing
2  X_train, X_test, y_train, y_test = train_test_split(
3      features_pca, target, test_size=0.2, stratify=target, random_state=42
4  )
5
6  smote = SMOTE(random_state=42)
7  X_train_bal, y_train_bal = smote.fit_resample(X_train, y_train)
8
9  print("Dataset split summary:\n")
10     print(f"Train(original):  {len(X_train):4d}   and   "f"Pass={{(y_train==0).sum()}}
11 Fail={{(y_train==1).sum()}}")
12     print(f"Train (SMOTE):  {len(X_train_bal):4d} and "f"Pass={{(y_train_bal==0).sum()}}
13 Fail={{(y_train_bal==1).sum()}}")
14     print(f"Test(unchanged):  {len(X_test):4d}   and"   f"Pass={{(y_test==0).sum()}}
15 Fail={{(y_test==1).sum()}}")
16
17     X_tr_sm, y_tr_sm = X_train_bal, y_train_bal
18     X_te, y_te = X_test, y_test

```

```

1  #k-Nearest Neighbours
2  k_candidates = list(range(3, 30, 2))
3  knn_grid = GridSearchCV(
4      KNeighborsClassifier(),
5      {'n_neighbors': k_candidates, 'weights': ['uniform', 'distance']},
6      cv=5, scoring='recall', n_jobs=-1
7  )
8  knn_grid.fit(X_tr_sm, y_tr_sm)
9  best_knn_k = knn_grid.best_params_['n_neighbors']
10 cv_f1_knn = [
11     knn_grid.cv_results_['mean_test_score'][
12         list(knn_grid.cv_results_['param_n_neighbors']).index(k)
13     ]
14     for k in k_candidates
15 ]
16 print("k-NN CV and Recall scores:")
17 for kv, fv in zip(k_candidates, cv_f1_knn):
18     marker = ' <= selected' if kv == optimal_k else ''
19     print(f" k={kv:>2} : Recall = {fv:.4f}{marker}")
20
21 #f1 vs K plot
22 fig, ax = plt.subplots(figsize=(8, 4))
23 ax.plot(k_candidates, cv_f1_knn, 'o-', color='#5C85D6', lw=2, ms=7)
24 ax.axvline(best_knn_k, color='green', ls='--', lw=1.5, label=f'Best k={best_knn_k}')
25 ax.set(title='k-NN: 5-Fold CV Recall vs k (Bias-Variance Trade-off)', xlabel='k',
26 ylabel='Mean CV Recall (Fail)')
27 ax.legend()
28 plt.tight_layout()

```

```

29 plt.show()
30
31 knn_clf = KNeighborsClassifier(n_neighbors=best_knn_k)
32 knn_clf.fit(X_tr_sm, y_tr_sm)
33 pred_knn = knn_clf.predict(X_te)
34
35 print(f"\nk-NN (k={best_knn_k})— Test Classification Report:")
print(classification_report(y_te, pred_knn, target_names=['Pass', 'Fail']))

```

```

1 #Decision Tree: Hyperparameter Tuning(max_depth) and Evaluation
2 dt_param_grid = {
3     'max_depth': [3, 5, 7, 10, 15, None],
4     'min_samples_leaf': [1, 3, 5, 10],
5     'criterion': ['gini', 'entropy'],
6     'class_weight': ['balanced']
7 }
8 dt_grid = GridSearchCV(
9     DecisionTreeClassifier(random_state=42), dt_param_grid,
10    cv=5, scoring='f1_macro', n_jobs=-1
11 )
12 dt_grid.fit(X_tr_sm, y_tr_sm)
13 best_depth = dt_grid.best_params_['max_depth']
14 print("Best DT params:", dt_grid.best_params_)
15
16 cv_f1_scores = [
17     dt_grid.cv_results_['mean_test_score'][
18         list(dt_grid.cv_results_['param_max_depth']).index(d)
19     ]

```

```

20     for d in [3, 5, 7, 10, 15, None]
21 ]
22 print("CV F1 scores by max_depth:")
23 depth_candidates = [3, 5, 7, 10, 15, None]
24 for d, score in zip(depth_candidates, cv_f1_scores):
25     selected = " <- best" if d == best_depth else ""
26     print(f"depth={str(d):>4} : {score:.4f}{selected}")
27 #final model
28 dt_model = dt_grid.best_estimator_
29 dt_model.fit(X_tr_sm, y_tr_sm)
30 y_pred = dt_model.predict(X_te)
31
32 print(f"\nTest Results (Decision Tree, max_depth={best_depth}):")
33 print(classification_report(y_te, y_pred, target_names=['Pass', 'Fail']))

```

```

1 #SVM
2     svm_params = { "C": loguniform(0.1, 100),"gamma": loguniform(1e-4,
3 1e-1),"kernel": ["rbf", "sigmoid"]}
4
5 svm_search = RandomizedSearchCV(
6     SVC(class_weight="balanced", random_state=42),
7     param_distributions=svm_params,
8     n_iter=30,
9     cv=5,
10    scoring='f1_macro',
11    n_jobs=-1,
12    random_state=42
13 )

```

```

14
15 svm_search.fit(X_tr_sm, y_tr_sm)
16 best_params = svm_search.best_params_
17
18 print(
19     f"Best SVM parameters: "
20     f"C={best_params['C']:.4f}, "
21     f"gamma={best_params['gamma']:.6f}, "
22     f"kernel={best_params['kernel']}"
23 )
24
25 #Final tuned SVM model
26 svm_model = svm_search.best_estimator_
27
28 y_pred_svm = svm_model.predict(X_te)
29
30 print("\nTest Results (Tuned SVM):")
    print(classification_report(y_te, y_pred_svm, target_names=["Pass", "Fail"]))

```

7. Model Evaluation

```

1 #Confusion Matrices
2 all_predictions = {'k-NN': pred_knn, 'Decision Tree': y_pred, 'SVM': y_pred_svm}
3 fig, axes = plt.subplots(1, 3, figsize=(15, 4))
4 for ax, (model_name, y_predicted) in zip(axes, all_predictions.items()):
5     ConfusionMatrixDisplay.from_predictions(
6         y_te, y_predicted, display_labels=['Pass', 'Fail'], ax=ax, colorbar=False,
7         cmap='Blues')
8     recall_val = recall_score(y_te, y_predicted, zero_division=0)
9     f1_val = f1_score(y_te, y_predicted, zero_division=0)

```

```

10     ax.set_title(f'{model_name}\nRecall (Fail) = {recall_val:.2f} | F1 (Fail) =
11 {f1_val:.2f}', fontweight='bold')
12
13     plt.suptitle('Confusion Matrices for all three Classifiers', fontsize=13,
14 fontweight='bold', y=1.03)
15
16     plt.tight_layout()
17
18     plt.savefig('confusion_matrix.png', dpi=300, bbox_inches='tight')
19
20     plt.show()

```

```

1  #comprehensive metrics table
2  metrics_rows = [
3      evaluate_classifier(name, y_te, y_pred)
4      for name, y_pred in all_predictions.items()
5  ]
6  metrics_df = pd.DataFrame(metrics_rows).set_index('Model')
7
8  print("Model Comparison Table:")
9  print(metrics_df.to_string(float_format='{:.4f}'.format))

```

```

1  #model performance comparison bar chart
2  fig, ax = plt.subplots(figsize=(13, 5))
3  metrics_df.plot.bar(ax=ax, edgecolor='white', width=0.75)
4  ax.set(title='Model Performance Comparison: All Metrics', ylabel='Score', ylim=(0,
5  1.12))
6  ax.set_xticklabels(metrics_df.index, rotation=0)
7  ax.axhline(1.0, color='grey', ls=':', lw=1)
8  ax.legend(loc='upper right', fontsize=9)

```

```

9
10 for i, (row_name, row_vals) in enumerate(metrics_df.iterrows()):
11     ax.text(i, row_vals['F1 (Macro)'] + 0.01, f"F1={row_vals['F1 (Macro)']:.3f}",
12     ha='center', fontsize=8, fontweight='bold')
13
14 plt.tight_layout()
    plt.savefig('model_performance.png', dpi=300, bbox_inches='tight')
    plt.show()

```

```

1 #5-Fold Cross-Validation Comparison
2 print("5-Fold CV F1(macro) on balanced training data:\n")
3 results = {}
4 models = {
5     'k-NN': knn_clf,
6     'Decision Tree': dt_model,
7     'SVM': svm_model
8 }
9 for name, model in models.items():
10     scores = cross_val_score(model, X_tr_sm, y_tr_sm, cv=5, scoring='f1_macro')
11     results[name] = {'F1 mean': scores.mean(), 'F1 std': scores.std()}
12     print(f"{name:15} | F1={scores.mean():.4f} ±{scores.std():.4f}")

```

```

1 original_models = {
2     "k-NN(original)": KNeighborsClassifier(n_neighbors=7),
3     "Decision Tree(original)": DecisionTreeClassifier(max_depth=5,
4 class_weight='balanced', random_state=42),
5     "SVM (original)": SVC(kernel='rbf', C=1.0, gamma='scale', class_weight='balanced',
6 random_state=42)
7 }
8
9 tuned_predictions = {"k-NN (tuned)": pred_knn,
10 "Decision Tree (tuned)": y_pred,
11 "SVM (tuned)": y_pred_svm}
12
13 print(f"{'Model':<25} | {'Recall Fail':>11} | {'F1 Fail':>8} | {'F1 Macro':>8}")
14 print("-" * 62)
15
16 #original scores
17 for name, model in original_models.items():
18     model.fit(X_tr_sm, y_tr_sm)
19     pred = model.predict(X_te)
20     f1_mac = f1_score(y_te, pred, average='macro')
21     f1_fail = f1_score(y_te, pred, zero_division=0)
22     rec = recall_score(y_te, pred, zero_division=0)
23     print(f"{'name':<25} | {'f1_mac':>8.4f} | {'f1_fail':>8.4f} | {'rec':>11.4f}")
24
25 print("-" * 62)
26
27 #tuned scores
28 for name, pred in tuned_predictions.items():

```

29	<code>f1_mac = f1_score(y_te, pred, average='macro')</code>
30	<code>f1_fail = f1_score(y_te, pred, zero_division=0)</code>
	<code>rec = recall_score(y_te, pred, zero_division=0)</code>
	<code>print(f"{name:<25} {rec:>11.4f} {f1_fail:>8.4f} {f1_mac:>8.4f}")</code>